

Legion — evidence and provenance

Every measured number, every code defect, every decision

What this is

The provenance layer: every measured number, every code defect, every decision. **This is reference, not a read.** Scan the findings index in Reference 1 and then look up only what you need — most likely because someone in the meeting asked "how do you know that?".

	Reference	Source	Lines
1	Every number and defect — R1-R6, D1-D11, M1-M2	PUBLICATION_MATERIALS_legion.md	361
2	What was done, round by round	WORKLOG_legion.md	103
3	The three days, hour by hour	rounds/Round-2026-08-27-...md	148
4	Nine decisions, each with what it rules out	decisions_legion.md	124
5	The Gymnasium GPU re-test in detail	gpu-retest-2026-08-29.md	80
6	Four Isaac Lab built-in tasks	isaaclab-builtin-tests-2026-08-29.md	67
7	The 1 to 128 environment sweep	dt-via-isaaclab-2026-08-29.md	67

Single-sourcing rule. A measured number or a code defect lives in Reference 1 and nowhere else. If any other document disagrees with it, Reference 1 wins.

The five findings most likely to be challenged

- **D10 — the handle is unreachable.** The blocking one. 0.834 m to the handle, ~0.85 m of reach, 0.207 m closest approach over 300 sampled arm poses, base DOF not actuated, start pose fixed.
- **D9 — replay-buffer sampling is O(buffer).** `np.random.choice(..., replace=False)` permutes the whole buffer every call: 460× slower than `randint` at 200k entries, with the GPU at 3% while the CPU pegged. **This retroactively affects the published Gymnasium results** — short runs never fill the buffer enough to notice, long ones were throttled.
- **D1 — the TensorFlow/Triton conflict.** Two lines reproduce a segfault. The silent second symptom is worse: importing TF makes `torch.cuda.is_available()` return `False`, so torch agents have been running on CPU while reporting nothing wrong.
- **D11 — the fingertip prims do not resolve.** `left_inner_finger` and `right_inner_finger` return the same world transform as `wrist_3_link`, so the straddle test is `x > h AND x < h` — false for every input — which gated every door-opening reward term to zero.
- **M1 — why these numbers are not comparable to the published ones.** Different machine, CPU vs GPU, and the driver seeds from wall clock so nothing is reproducible either way. Read this before putting an old and a new table on the same slide.

Two caveats that apply to nearly every number here

Single seed, and the driver seeds from wall-clock time. Nothing in this packet is reproducible run to run, in either direction. Getting to three seeds with reported variance is the largest outstanding weakness in the project's evidence.

Throughput figures are order-of-magnitude. Two episodes per configuration, with large per-episode variance — 419 then 134 env-steps/s at the same 64 environments.

Generated 2026-09-02 from RL_DT@d827a53, which is the commit that pulled the Legion work onto the PNLL machine. Assembled verbatim from the markdown sources named in each section header; nothing was rewritten for this packet, and the framing notes under each header are the only added text. Chromium print does not emit page numbers, so the section banners are the navigation.

Reference 1 — every number and every defect

RL_DT/_docs_legion/PUBLICATION_MATERIALS_legion.md - 362 lines

The single home for all of it: 6 results R1-R6, 11 defects D1-D11, 2 method notes. Scan the findings index, then read only what you need. Do not re-derive a number from anywhere else — if another document disagrees with this one, this one wins.

Publication materials — work on trossen-ai (the Legion machine)

Companion to PUBLICATION_MATERIALS.md, which is **not modified**. Same rule applies within this file: a measured number or a code defect lives here once, and other _legion documents link to it.

All results 2026-08-27 → 2026-08-29, RTX 5090 Laptop (Blackwell, sm_120), torch 2.13.0+cu130.

Findings index

Results - R1 — Gymnasium environments re-tested on GPU - R2 — Toolkit drives four Isaac Lab built-in environments - R3 — Digital twin via Isaac Lab, scaling 1→128 environments - R4 — Digital twin via bare Isaac Sim - R5 — Route comparison - R6 — Multi-hour learning runs: neither route learns the task

Code defects - D1 — TensorFlow/Triton native-library conflict (segfault) - D2 — setup.py omits every subpackage - D3 — Three subpackages missing __init__.py - D4 — gymnasium-robotics missing from requirements - D5 — test_torch_td3.py assumes CPU tensors - D6 — FetchReach incompatible with installed MuJoCo - D7 — Door scene referenced by bare filename - D8 — Stale absolute Windows path inside the USD - D9 — Replay-buffer sampling is O(n) in buffer size - D10 — Door handle is unreachable with the actions Martin's task exposes - D11 — Fingertip prims do not resolve their own transforms

Method notes - M1 — Why these numbers are not comparable to the published ones - M2 — Isaac Sim cannot recreate environments in one process

Results

R1 — Gymnasium environments re-tested on GPU

TorchTD3-v0, each environment at its originally published episode/step counts, after D1 made the GPU usable. All reported Device: cuda.

Environment	Episodes × steps	Final avg-20	Wall time
Pendulum-v1	100 × 200	−160.30	111 s
MountainCarContinuous-v0	100 × 999	−1.02	477 s
Hopper-v5	300 × 1000	187.10	71 s
AdroitHandDoor-v1	20 × 200	−42.31	23 s
FetchReachDense-Flat-v0	500 × 50	did not start — D6	—

A separate 100-episode Pendulum-v1 run measured the learning curve: crosses avg-20 of −200 at episode 61, ends at −132.3 in 5 m 20 s. Curve every 10 episodes: −1226, −1221, −1320, −1325, −727, −253, −233, −152, −128, −150.

For context only, from the published (CPU, different machine) results: the Keras baseline crosses at episode 47 and ends at −163.0. See M1 before comparing.

Evidence: _toolkits/toolkit-isaacclab/results/<env>/index090_*

R2 — Toolkit drives four Isaac Lab built-in environments

Same isaac_vec_adapter.py used for Martin's door task, no per-task code, --num_envs 16 --steps 30.

Task	Obs	Actions	Throughput
Isaac-Cartpole-v0	4	1	659 env-steps/s
Isaac-Ant-v0	60	8	440 env-steps/s
Isaac-Reach-Franka-v0	32	7	257 env-steps/s
Isaac-Open-Drawer-Franka-v0	31	8	364 env-steps/s

4/4 passed. Observation dimension spans 4→60 and actions 1→8. This is the evidence that the adapter is a general toolkit↔Isaac Lab bridge rather than something shaped around one scene.

Rewards are recorded in isaac_integration/builtin_results.txt but are **not** performance figures — 30 steps of a barely-initialised policy. They show only that the reward path is live.

Isaac-Open-Drawer-Franka-v0 is NVIDIA's cabinet task: articulated object, grasp a handle, pull it open — structurally the same problem as Martin's door, and probably what it was modelled on.

R3 — Digital twin via Isaac Lab, scaling 1→128 environments

Isaac-UR5e-DoorOpen-v0, obs 43, actions 7, 2 episodes × 40 steps per configuration.

--num_envs	Env-steps/s (ep 0, ep 1)	Speed-up
1	15, 17	baseline
16	191, 180	~11×
64	419, 134	~9–25×
128	271, 181	~12–17×

Throughput climbs steeply to 16 environments then flattens. The plateau is explained: the simulator steps N environments together on the GPU, but **action selection is one forward pass per environment in a Python loop**, so past some N that loop dominates rather than the physics.

Per-episode variance is large (419 then 134 at 64 envs). Two episodes per configuration is not enough to average out first-touch shader compilation. Treat as order-of-magnitude.

The more important consequence than speed: `cfgs/torch_td3.cfg` sets `warmup_size: 2500`. TD3 takes random actions until 2500 transitions are buffered. At 1 environment a 40-step episode contributes 40 transitions; at 128 it contributes 5120. **The single-environment configuration cannot practically reach the point where learning begins.** Parallelism here is a correctness requirement, not only a speed optimisation.

R4 — Digital twin via bare Isaac Sim

No Isaac Lab, no Docker. `door_env_direct.py`, 345 lines written from scratch.

Observation dim	35 (assembled here; not the same as Isaac Lab's 43)
Action dim	7 (6 UR5e arm joints + gripper)
Device	<code>cuda</code>
Throughput	130–200 steps/s, single environment

Verified against real physics rather than assumed: - All 8 prim paths from Martin's config resolve - Robot articulation exposes 15 DOF with the expected joint names - `/World/OpenTronsFlex` is itself a 2-DOF articulation containing `Door_RevoluteJoint` - Commanding the hinge to -30° , -90° , -170° reads back -29.5° , -88.3° , -167.9° — so observation, reward and success test are wired to live state

Evidence: `_toolkits/toolkit-isaacsim/_isaacsim_notes/`.

Scope limit: the reward here is three terms (approach / opening / effort). Martin's task encodes roughly ten shaped terms across 1140 lines, tuned together. This is a structurally equivalent task, **not** a reproduction.

R5 — Route comparison

Matched: 1 environment, 2 episodes × 40 steps.

	Bare Isaac Sim	Isaac Lab + Docker
Throughput, 1 env	148–151 steps/s	15–17 steps/s
Best measured	148–151 (1 env only)	~419 (64 envs)
Parallel environments	1	1→128 tested
Code written	345 lines	248 lines
Task definition reused	none	1140 lines (Martin's)
Disk	18 GB	34.7 GB image + 23 GB base
Runs without Docker	yes	no
GUI	yes, verified	awkward from container

The single-environment gap flatters the direct route: that overhead is Isaac Lab doing the work that makes many environments possible. See `comparison-isaacsim-vs-isacclab.md`.

Neither route has been shown to learn the door task. All runs were minutes long with essentially untrained policies. Nothing here supports a claim about learning performance, and the two routes do not even use the same reward.

R6 — Multi-hour learning runs: neither route learns the task

Full write-up: `long-run-learning-2026-08-30.md`.

	Bare Isaac Sim (base actuated, 10 actions)	Isaac Lab (Martin's config, 7 actions)
Episodes	~1,610	~210 x 64 envs
avg-20 reward	105.8 -> 334.3 (peak 338.4)	5.85 -> 9.38
Plateaued	~episode 400	immediately
Door opened	2/162 sampled episodes (1.2%)	never

The trained policy, replayed deterministically without exploration noise, never moves the door - 6 episodes out of 6. The 1.2% of training episodes where the door moved were exploration noise shoving it, not learned behaviour; the peak of exactly 3.1416 rad is the hinge's -180 deg limit, the signature of the door being knocked to its stop.

So the tripled reward is real optimisation of the approach and alignment terms, and **not** progress on the task. Reported as such: claiming "learned to open the door" would have been wrong, and wrong in the specific way the brief's no-brute-force-pushing requirement exists to prevent.

The Isaac Lab route's flat 9.38 is the *correct* outcome given D10, not a training failure.

Code defects

D1 — TensorFlow/Triton native-library conflict

Severity: blocking. TorchTD3-v0 segfaulted before a single training step.

TensorFlow and Triton (PyTorch's GPU compiler, loaded lazily when an optimizer is constructed) ship conflicting native libraries. **Whichever loads second segfaults.** The toolkit's import order loaded TF first — via the Keras agents, and via `torch.utils.tensorboard`, which imports TF — so Triton lost.

Two-line reproduction:

```
import tensorflow          # then
import triton._C.libtriton # -> SIGSEGV
```

Reverse the order: both load fine.

Second symptom, independent of the crash: importing TF installs CUDA stub libraries, after which `torch.cuda.is_available()` returns `False` with *"Error 302: Error loading CUDA libraries"*. The torch agents therefore ran on **CPU while reporting no error**, even on a GPU machine.

Fix: 5 files, 191 lines, in `_patches/toolkit-tf-triton-fix.patch`. The core is pre-loading `triton._C.libtriton` at package import, guarded by `if 'tensorflow' not in sys.modules` — if the caller already imported TF, forcing Triton in afterwards is the crashing order.

Side effect: the lazy-import work cleared the blocker recorded in `strategy.md` Part 2 ("the package will not import in a torch-only container"). The Isaac Lab container ships PyTorch but no TensorFlow, so this is what made the digital-twin wiring possible at all.

D2 — `setup.py` omits every subpackage

`packages=['jlab_opt_control']` lists no subpackages. A strict editable install (`pip install -e .`) therefore exposes only the top-level package, and `import jlab_opt_control.buffer`s fails with `ModuleNotFoundError`. Unnoticed upstream because running from the repo root picks the subpackages up via the current directory instead. Fixed with `find_packages()` plus `package_data` for `cfigs/*.cfig`.

D3 — Three subpackages missing `__init__.py`

`buffer`s/, `driver`s/ and `util`s/ have no `__init__.py`, so they are implicit namespace packages and `find_packages()` skips them even after D2 is fixed. Added.

D4 — `gymnasium-robotics` missing from requirements

Required by `envs/robotics_flat.py` and by `utests/test_registry.py::test_env`, but in neither `requirements.txt` nor `env.yaml`. Same category as the `pysindy / scikit-learn` gap already noted in `environment.md`. Installed here as 1.4.2 — but see D6 before pinning.

D5 — `test_torch_td3.py` assumes CPU tensors

With D1 fixed, 12 tests fail with *"Expected all tensors to be on the same device, but found cuda:0 and cpu"*. The tests build plain CPU tensors (`torch.randn(...)`, no device) while the agent's networks now correctly live on the GPU. Before the fix, TF broke CUDA detection so the agent silently fell back to CPU and everything accidentally matched.

Not a regression: on the unmodified toolkit these same tests *segfault* — they could not run here at all. And forcing `"device": "cpu"` in `cfigs/torch_td3.cfig` makes **all 47 pass**, confirming the diagnosis. The agent is correct; the tests are not GPU-aware, having been written on a CPU-only laptop.

Deliberately **not** fixed here — it is a design choice for Malachi: make the tests device-aware (keeps GPU coverage, more edits) or pin them to CPU (smaller, leaves the GPU path untested).

Full suite after the fix: **281 passed, 13 failed** (12 of these + D6), 10 skipped, 39 min.

D6 — FetchReach incompatible with installed MuJoCo

```
gymnasium_robotics/utils/mujoco_utils.py:142 in set_joint_qpos
    assert joint_type in (mujoco.mjtJoint.mjJNT_HINGE, mujoco.mjtJoint.mjJNT_SLIDE)
AssertionError
```

Fails inside gymnasium-robotics' own Fetch setup, before the toolkit is involved — the environment cannot be constructed. Versions: gymnasium-robotics 1.4.2, mujoco 3.12.0, gymnasium 1.3.0.

Fetch-specific: AdroitHandDoor-v1 constructs and trains on the same install, so envs/robotics_flat.py is not implicated. Untested fixes: pin an older gymnasium-robotics, or an older mujoco. Worth resolving if the published FetchReach numbers need reproducing.

D7 — Door scene referenced by bare filename

ur5e_door_open_env_cfg.py had USD_PATH = "TrainingScene_flattened.usd" while the file lives in a sibling directory, so Isaac Lab failed with FileNotFoundError. Changed to "../ENV_OT_UR_door/TrainingScene_flattened.usd". The line sits under a comment reading "User-editable paths", so per-setup adjustment appears expected rather than a defect. **Reapply if Martin's repo is ever re-synced.**

D8 — Stale absolute Windows path inside the USD

TrainingScene_flattened.usd carries a payload reference to file:/C:/Users/prat615/Documents/ClonedProjects/digital_twin_models_1/Labware/OpentronsDoorAdapter/OpentronsDoorAdapter_large.usdc, which produces an alarming warning on every load.

It is a red herring. The prim /World/OpenTronsFlex/OpentronsDoorAdapter_large/LooRoll/Handle — the exact path the task's reward depends on — is present and valid, with its mesh and revolute joint, because the geometry was baked in when the scene was flattened. Cosmetic only, but worth documenting so it does not cause a false alarm later (it caused one here).

D9 — Replay-buffer sampling is O(n) in buffer size

Severity: throttles every long run. er.py sampled with np.random.choice(max_index, size=256, replace=False). NumPy implements that by building and permuting an array of the whole buffer per call:

buffer	choice(replace=False)	randint	penalty
1,000	0.40 ms	0.03 ms	12x
50,000	4.80 ms	0.04 ms	112x
200,000	19.88 ms	0.04 ms	460x

Observed live: the direct route fell from 150 to 10.8 steps/s at buffer 56k, the Isaac Lab route from 195 to 21 env-steps/s, **with the GPU at 3% and the CPU pegged**. Fixed by switching to np.random.randint - sampling with replacement, as SB3, CleanRL and the reference TD3 implementation all do. Measured 5-6 ms -> 0.050 ms at 60k entries.

Not Isaac-specific: affects every long run in the toolkit, including the published Gymnasium results. Short runs never fill the buffer enough to see it.

D10 — Door handle is unreachable with the actions Martin's task exposes

Severity: the task cannot be solved as configured.

```
handle -> arm base distance      0.834 m   (UR5e reach ~0.85 m)
closest gripper->handle, 300 random arm poses  0.207 m
closest, 400 poses WITH base translation  0.137 m
```

ActionsCfg exposes arm_action and gripper_action only - the Ridgeback's three base DOF are not actuated - and EventCfg sets position_range: (0.0, 0.0), so every episode starts from the same out-of-reach pose. No policy can open the door.

This is the explanation for R6's flat Isaac Lab curve. **A question for Martin:** is the base meant to be actuated, or the robot meant to be parked within reach? The direct route was given the base joints (action dim 7 -> 10) to work around it.

D11 — Fingertip prims do not resolve their own transforms

left_inner_finger and right_inner_finger report the same world transform as wrist_3_link - verified via XFormPrim, RigidPrim and USD's ComputeLocalToWorldTransform, and confirmed to move with the arm but never to separate from the wrist.

Martin's grasp test is lfinger_z > handle_z AND rfinger_z < handle_z, which with identical values is x > h AND x < h: false always. In the ported reward this zeroed two terms outright, and because the three opening terms are gated on the grasp state, **every door-opening term was permanently zero as well** - there was no gradient toward opening the door. Substituted a TCP-height proxy.

Whether the same issue affects Isaac Lab's FrameTransformer-based version of these terms was not tested, and is worth checking - if it does, Martin's own reward has dead terms too.

Method notes

M1 — Why these numbers are not comparable to the published ones

Three independent reasons, any one of which is sufficient:

1. **The published runs were CPU runs on a different machine** (Dewei's laptop). Additionally, before D1 was fixed, *any* torch run silently fell back to CPU regardless of hardware.
2. **The driver seeds from wall-clock time** (`run_continuous.py`), so no run is reproducible — old or new. Single seed per arm, as the roadmap already flags.
3. **Library versions differ**. Specifically, gymnasium-robotics warns that `AdroitHandDoor`'s reward function changed in 1.2.1 **without an environment version bump**, so the `-42.31` in R1 may be measuring a different reward than the published figure.

Read R1 as *"the toolkit trains correctly on this machine"*, not as a benchmark comparison.

M2 — Isaac Sim cannot recreate environments in one process

A test that looped over four tasks in a single process — `env.close()` then `gym.make()` for the next — **hung for 7 hours 40 minutes**: GPU pinned at 0% utilisation with 2.5 GB VRAM held, process sleeping, no output after the first task.

Isaac Sim does not reliably support tearing down an environment and creating another within one session. The loop must live in the shell: one process per task, with a `timeout` on each. Restructured that way, the same four tasks finish in minutes.

Two operational lessons: never loop over Isaac Sim environment creation in-process, and always put a timeout on a long-running background job — the 7h40m was invisible precisely because nothing would ever have stopped it.

Reference 2 — what was done, round by round

RL_DT/_docs_legion/WORKLOG_legion.md - 104 lines

Two entries. The newest is the learning runs; below it the three-day integration round.

WORKLOG — trossen-ai (the Legion machine)

Companion to WORKLOG.md, which is **not modified** — it records work from Dewei's laptop and the PNNL workstation. This file records work done on the Legion machine, which is a separate environment with different capabilities (no tanuki access, an RTX 5090, Isaac Sim installed two ways).

Same conventions: newest round at the top, about six bullets each, detail in rounds/. Measured numbers and code defects go to PUBLICATION_MATERIALS_legion.md, not here.

Index

Round	Goal	Status
Round-2026-08-27	Stand the project up on a new machine, then connect the RL toolkit to the digital twin	done — three integration routes working, toolkit fixed for Blackwell GPUs, Isaac Sim installed both ways

2026-08-30 → 08-31 — Do the routes actually learn? No, and here is why

Multi-hour runs on both routes to answer the question every earlier result left open. Full report: long-run-learning-2026-08-30.md.

- **Neither route learned the door task.** The direct route's reward tripled (105.8 → 334.3) and converged by episode ~400, but replaying the trained policy deterministically moves the door in **0 of 6 episodes**. The 1.2% of training episodes where the door moved were exploration noise shoving it — the peak of exactly 3.1416 rad is the hinge's ~180° limit, not a pulled-open door.
- **The task is not solvable as configured**, and this was measured rather than inferred: the handle sits 0.834 m from the arm base against a ~0.85 m reach, and the gripper gets no closer than 0.207 m over 300 sampled arm poses. Martin's ActionsCfg actuates arm and gripper only. D10 — **needs a decision from Martin**.
- **Found an upstream performance bug that throttles every long run:** the replay buffer sampled with np.random.choice(..., replace=False), which is O(buffer). 460× slower than randint at 200k entries; the GPU sat at 3% while the CPU pegged. D9.
- **Three mistakes of my own**, all caught and documented: half the ported reward was silently dead because the fingertip prims do not resolve (D11); neither training script logged reward at all, which a pilot caught before the night was wasted; and a .gitignore pattern aimed at buffer data also hid the buffer source package from git.
- **Logging door angle, not just reward, is what made the diagnosis possible** — reward rising while door angle stayed at zero is precisely the signature of optimising shaped terms without solving the task.

2026-08-27 → 2026-08-29 — New machine, toolkit fix, and three routes to the digital twin

Started from a copied project folder on an unfamiliar machine and ended with the toolkit driving Martin's door task through two independent stacks. The bulk of the effort was not the integration — it was making the toolkit run at all.

- **Machine brought up.** GPU was dead on arrival (nvidia-smi failing, no kernel module for the running kernel); resolved by pinning the boot kernel. Docker, git-lfs and uv installed. This machine cannot reach tanuki or the PNNL shared drives, so both DT repos exist as extracted copies and the Isaac Lab image was built locally rather than pulled. See environment_legion.md.
- **The blocker: TorchTD3-v0 segfaulted before its first training step.** Diagnosed to a TensorFlow/Triton native-library conflict — whichever loads second crashes — plus a silent second symptom where importing TF makes torch stop seeing the GPU. Fixed in 5 files; the Keras agents were regression-tested and still train. D1, patch in ../_patches/. **An initial diagnosis blaming an upstream PyTorch/Blackwell bug was wrong** and is recorded as such.
- **Side effect worth more than the fix:** the lazy-import work cleared the blocker in strategy.md Part 2 — the package now imports in a torch-only container, which is what made everything Isaac-related possible.
- **Environments re-tested on GPU.** Four of five published environments reproduced; FetchReachDense-Flat-v0 cannot start due to a gymnasium-robotics / mujoco incompatibility unrelated to the toolkit. R1, D6.
- **Isaac Sim installed twice, deliberately** — the Docker container (Isaac Lab, for training) and a 5.1.0 workstation install (GUI, for looking at the twin). Both verified loading Martin's scene. Guides: ISAAC_SIM_UI_GUIDE.md, DIGITAL_TWIN_HOWTO.md.
- **Three integration routes built and measured** — bare Isaac Sim (345 lines, no framework), Isaac Lab single-env, and Isaac Lab vectorised (1→128 environments). The vectorised adapter also drove four Isaac Lab built-in tasks unchanged, which is what shows it is a general bridge rather than door-task-shaped. R2–R5.
- **Three upstream toolkit bugs found** beyond the segfault: packaging that breaks pip install -e ., three subpackages missing __init__.py, and an undeclared dependency. team-discussion_legion.md has the write-ups to send.

What this does not show: that anything *learns the door task*. Every run was minutes long with an untrained policy, and the two routes do not use the same reward. Stated at length in comparison-isaacsim-vs-isaaclab.md.

Documents produced this round

File	What it holds
session-2026-08-27-legion.md	The session summary, written for the team
PUBLICATION_MATERIALS_legion.md	Every measured number and code defect
environment_legion.md	Machine setup, kernel pin, network limits, both Isaac Sim installs
decisions_legion.md	Nine decisions with reasoning and what each rules out
team-discussion_legion.md	Items for Malachi, Martin and Alvika
ISAAC_SIM_UI_GUIDE.md	Exploring the twin in the GUI, for learning
DIGITAL_TWIN_HOWTO.md	Running the twin through Docker
comparison-isaacsim-vs-isaacsim.md	The two routes, head to head
gpu-retest-2026-08-29.md	The re-tested environments in detail
isaacsim-builtin-tests-2026-08-29.md	The four built-in tasks
dt-via-isaacsim-2026-08-29.md	The scaling sweep

That is more files than the six-live-file budget in `README.md` would suggest. They are kept separate because this machine's record should not be interleaved with the originals, but consolidating them into the main documents is worth doing once the two machines' work is merged.

Reference 3 — the three days, hour by hour

RL_DT/_docs_legion/rounds/Round-2026-08-27-legion-gpu-fix-and-dt-integration.md - 149 lines

Provenance. Worth reading one section for its own sake: 'Mistakes made in this round'.

Round 2026-08-27 → 2026-08-29 — Legion machine: GPU fix and digital-twin integration

Goal: stand the project up on a new machine, then connect the RL toolkit to the digital twin.

Status: done. Three integration routes working; the toolkit runs on Blackwell GPUs; Isaac Sim installed both as a container and as a workstation build.

Narrative record. Numbers and defects live in `../PUBLICATION_MATERIALS_legion.md`.

Day 1 (08-27) — the machine did not work

The project folder had been copied to `trossen-ai`, an Ubuntu box with an RTX 5090. Very little was runnable:

- `nvidia-smi` failed. The GPU was physically present and the driver installed, but **no kernel module existed for the running kernel**. The archive's module package for `6.17.0-35-generic` has a self-contradictory dependency that apt cannot resolve, so the fix was to default-boot `6.17.0-14-generic`, which already had one. That needed `GRUB_DEFAULT=saved` as well — `grub-set-default` alone silently does nothing while `GRUB_DEFAULT=0` is set.
- The toolkit's `.venv` was a **Windows** virtualenv, unusable on Linux. Rebuilt with `uv`.
- `git-lfs` was missing; `isaacsim-dt-main/docker/.env.base` pointed at another machine's paths.
- The machine **cannot reach tanuki or the PNNL shared drives** — internal DNS resolves but every port refuses. So both DT repos exist here as extracted copies rather than clones, and the Isaac Lab image had to be built locally instead of pulled from the GitLab registry.

A process note worth recording. Early in this round several changes were made under what read as a broad grant of overnight autonomy, and the user interrupted to stop it. The working mode for the rest of the round became: propose a bounded plan, get explicit approval, then execute — which is how everything after that point was done.

Day 2 (08-28) — the toolkit could not run at all

`TorchTD3-v0` segfaulted a few milliseconds after start, before a single training step.

A wrong turn first. The crash was traced to Triton, PyTorch's GPU compiler, and web search turned up genuine open PyTorch issues about Triton codegen segfaulting on `sm_120` (Blackwell) GPUs. That looked like a match, and it was written up as an unfixable-locally upstream bug. **It was wrong.**

What corrected it was the user pointing out that the same code runs fine on their other machines. A proper bisect then found the real cause: **TensorFlow and Triton ship conflicting native libraries, and whichever loads second segfaults**. Two lines reproduce it. The toolkit's import order loaded TF first — via the Keras agents, and via `torch.utils.tensorboard` — so Triton lost.

A second symptom turned out to matter independently: importing TF installs CUDA stubs that make `torch.cuda.is_available()` return `False`. **Torch agents had been running on CPU while reporting nothing wrong.**

The fix is five files: pre-load Triton at package import (guarded, so it does nothing if TF is already loaded), and make the Keras agents, models and the TF-backed `Model` base class import lazily. Keras agents were regression-tested and still train.

The side effect was worth more than the fix. Making the TF imports lazy cleared the blocker already recorded in `../strategy.md` Part 2 — *"the package will not import in a torch-only container"*. The Isaac Lab container has PyTorch but no TensorFlow at all. Without this, none of the Isaac work would have been possible.

Also on day 2: the Isaac Lab image was built locally (34.7 GB), the container started with GPU passthrough, and Martin's door task loaded and trained for one iteration — proving the digital twin itself works. One trivial fix was needed: the scene USD was referenced by bare filename while living one directory across.

A false alarm worth recording: the scene load warns about a missing `C:/Users/prat615/.../OpentronsDoorAdapter_large.usdc`. It looked serious — that is the door handle the robot must grasp — but the geometry was baked into the flattened USD and the prim is present and valid. Checked rather than assumed, after initially raising it as a problem.

Day 3 (08-29) — the seven-item list

With the toolkit working, the user set a list. Worked in the agreed order, committing after each.

Step 0 — capture the fix. The five-file change was uncommitted and existed only in a working directory. Saved as a standalone patch with a full write-up in `_patches/`.

#2 — Isaac Sim workstation install. A 5.0.0 build had been downloaded from a PNNL machine, but Alvika had flagged that 5.1.0 is required to match Martin's assets. 5.1.0 turned out to download directly from this machine with no NGC login, so no flash-drive transfer was needed. Installed to `/home/aidev/isaacsim`; verified it opens Martin's scene headlessly and that the **GUI reaches "app ready" in ~16 s with no errors**. Wrote

ISAAC_SIM_UI_GUIDE.md for exploring the twin visually, marking explicitly which steps were verified and which could not be (nothing here can see the screen).

#1 — re-test the published environments on GPU. Four of five reproduced. FetchReachDense-Flat-v0 cannot start: an assertion inside gymnasium-robotics' own Fetch setup against mujoco 3.12.0, before the toolkit is involved. Also found that AdroitHandDoor's reward function changed in gymnasium-robotics 1.2.1 *without a version bump*, so that number is not comparable to the published one without pinning the same library version.

#4 — toolkit against Isaac Lab built-ins. Four tasks, observation dimensions from 4 to 60, actions 1 to 8, all driven by the same adapter with no per-task code. This is the result that shows the adapter is a real bridge rather than something shaped around the door task.

A mistake of mine cost most of a day here. The first harness looped over all four tasks inside one Python process, closing and recreating environments. It **hung for 7 hours 40 minutes** — GPU at 0%, process asleep, and nothing would ever have stopped it because I had set no timeout. Isaac Sim does not support recreating an environment in one session. Restructured to one process per task with a 600 s timeout each; the same four tasks then finished in minutes.

#5 — digital twin via Isaac Lab, scaled. 1 → 128 environments. Throughput climbs ~11× to 16 environments then plateaus, because action selection is still one forward pass per environment in a Python loop while the simulator steps them all together.

#3 — the major task: digital twin via bare Isaac Sim. No Isaac Lab, no Docker. door_env_direct.py reimplements what Isaac Lab was providing — scene loading, observation assembly, action application, reward, termination, reset. It works: 35 observations, 7 actions, on the GPU.

Rather than trust that "no crash" meant "correct", the signal path was verified against physics: every prim path resolves, the articulation exposes 15 DOF with the expected names, and commanding the hinge to −30°/−90°/−170° reads back within 2°. That check caught a real bug — **the door opens in the negative direction**, so the first implementation scored a fully-opened door as zero reward.

Scope stated rather than glossed: the reward here is three terms against Martin's roughly ten tuned terms across 1140 lines. Structurally equivalent task, not a reproduction.

#6 — comparison. Bare Isaac Sim is ~10× faster per step at one environment, but that gap is Isaac Lab doing the work that makes many environments possible; Isaac Lab at 64 environments is ~3× the direct route's best. The decisive point is not speed: warmup_size is 2500, so a single-environment configuration **cannot practically reach the point where TD3 starts learning**. Recommendation is to keep both routes — Isaac Lab for training, bare Isaac Sim for GUI exploration and as the starting point if real hardware becomes the target.

#7 — sync. Dewei's own step. RL_DT now pushes to a private personal GitHub repo, since tanuki is unreachable from here.

Mistakes made in this round

Recorded because they cost real time and the reasoning is worth not repeating:

1. **Diagnosed the segfault wrong** and wrote it up confidently as an upstream PyTorch bug. Corrected only because the user said it works elsewhere. Lesson: when a user reports different behaviour on another machine, bisect rather than pattern-match to a plausible open issue.
2. **A 7h40m hang** from looping Isaac Sim environment creation in one process, made invisible by setting no timeout on the background job.
3. **Deleted jlab_opt_control/buffers/** with an rsync --exclude='buffers' intended for result data, which also matched the source package. Caught by a file-count comparison against the source.
4. **Raised a false alarm** about the missing door-handle asset before checking whether the prim actually resolved.

Open at the end of this round

- Neither route has been shown to **learn** the door task. Every run was minutes long.
- batch_action on the agent is the highest-value next change — it is what caps throughput on both routes, and it is a change to Malachi's code rather than a wrapper.
- The kernel pin is temporary and should be revisited when Ubuntu ships a matching NVIDIA module.
- Three toolkit bugs are written up but not sent upstream — see ../team-discussion_legion.md.
- FetchReach remains unreproducible on this machine.

Reference 4 — nine decisions, each with what it rules out

RL_DT/_docs_legion/decisions_legion.md - 125 lines

Read this when you want to reopen something. Two of the nine now affect this machine: sync via GitHub rather than tanuki, and untracking CLAUDE.md.

Decisions — trossen-ai (the Legion machine)

Companion to `decisions.md`, which is **not modified**. Append-only, newest last. Each entry: what was chosen, why, and what it rules out.

2026-08-27 — Pin the kernel rather than chase the NVIDIA module

Chose: default-boot 6.17.0-14-generic instead of the newest 6.17.0-35-generic.

Why: no NVIDIA kernel module exists for -35, and the archive's module package has a self-contradictory dependency on `nvidia-kernel-common-580` that apt cannot resolve. The older kernel already has a working module. Point releases within the same series differ little for a workstation.

Rules out: kernel security updates beyond -14 until this is revisited. **This is temporary** — check periodically whether a matching module ships for a newer kernel. Details in `environment_legion.md`.

2026-08-28 — Fix the TF/Triton conflict by import ordering, not by pinning versions

Chose: pre-load Triton's native extension at package import, guarded so it does nothing when TensorFlow is already loaded; plus lazy imports for the Keras agents, models and `Model` base class.

Why: the conflict is a load-order problem, not a version problem — either library works fine if it loads first. Downgrading torch or TF would have been a larger, more disruptive change with its own compatibility risk, and would not have helped the torch-only container case.

Rules out: nothing permanently; the patch is standalone in `_patches/` and reversible. Note the guard is essential — without it, a caller that imports TF first (as several unit tests do) crashes.

Side benefit: cleared the blocker in `strategy.md` Part 2 about the package not importing in a torch-only container.

2026-08-28 — Do not fix `test_torch_td3.py`'s CPU assumption

Chose: document the 12 device-mismatch test failures rather than edit the tests.

Why: the agent is behaving correctly (it now genuinely uses the GPU); the tests assume CPU tensors because they were written on a CPU-only laptop. There are two defensible fixes — make the tests device-aware, or pin them to CPU — with a real trade-off (GPU coverage vs. minimal change). That is Malachi's call on his own test suite, not a unilateral edit from here.

Rules out: nothing. Recorded as D5 for the conversation.

2026-08-29 — Install Isaac Sim 5.1.0, not the 5.0.0 that was downloaded

Chose: download and install 5.1.0 directly on this machine.

Why: Martin's USD assets were authored in 5.1, and the Docker container is pinned to 5.1.0. A 5.0.0 workstation install would be *older* than the assets — the riskier direction for USD schema mismatches — and would mean two different Isaac Sim versions reading the same files. Alvika had specifically flagged selecting 5.1.0 over the download page's default. 5.1.0 turned out to be downloadable directly from this machine, so no flash-drive transfer was needed.

Rules out: nothing. The 5.0.0 zip is untouched in `~/Downloads` if it is ever wanted.

2026-08-29 — Keep both integration routes rather than choosing one

Chose: maintain the bare-Isaac-Sim route and the Isaac Lab + Docker route in parallel, in separate toolkit copies.

Why: they serve different jobs and are not really competing. R5 shows the direct route is faster per step at one environment but cannot scale without reimplementing the parallelism Isaac Lab provides — and R3 shows a single environment cannot even reach TD3's `warmup_size`, so it cannot train. Conversely the direct route has a working GUI, no Docker dependency, and is closer to a hardware-style control loop if the real robot becomes the target.

Rules out: consolidating on one route for now. Revisit if the direct route gains parallelism, or if the hardware target firms up.

2026-08-29 — Separate toolkit copy per integration approach

Chose: `_toolkits/toolkit-isaacsim/` and `_toolkits/toolkit-isaacsim/`, each a full copy without git history, line endings normalised to LF.

Why (Dewei's reasoning): the approaches will diverge — different imports, different device handling, possibly incompatible adapters — and forcing one copy to serve both means constant merge pain for no benefit. Separate copies are isolation, not duplication. Copies were taken from the *working* toolkit rather than the pristine USB copy because the TF/Triton fix is required for the toolkit to run on this machine at all; the standalone patch keeps the delta from upstream explicit.

Rules out: a single shared copy. Note this puts toolkit code inside `RL_DT`, which the root `CLAUDE.md` rule ("*toolkit code goes there, notes go here*") would otherwise forbid — a deliberate exception, made viable because the upstream toolkit is public on GitHub.

2026-08-29 — Untrack `CLAUDE.md`

Chose: `git rm --cached CLAUDE.md` plus a `.gitignore` entry, so each machine keeps its own.

Why: it is machine-specific — paths, remotes, and GPU details differ between this box and the PNNL workstation, and the PNNL machine can reach tanuki while this one cannot.

Rules out: a shared project map. **Cost worth watching:** `CLAUDE.md` is the file every session reads first, so the two machines' maps will now drift silently. If that becomes a problem, the alternative is a shared `CLAUDE.md` plus a gitignored `CLAUDE.local.md` for machine specifics.

2026-08-29 — Sync via a private personal GitHub repo

Chose: `github.com/Dewei-Wang-xx/RL4DT-Toolkit` (private) as the transport between this machine and the PNNL computers.

Why: this machine cannot reach tanuki or the PNNL shared drives (see `environment_legion.md`), OneDrive login fails here, and USB is inconvenient for routine syncing. Git also gives history and honest diffs, which a file share would not.

Rules out: nothing technically. **Standing caveat:** this places PNNL work and colleagues' code on personal infrastructure. The repo is private and the upstream toolkit is public, which limits the exposure, but where the work should ultimately live is a question for Dewei, Malachi and Martin rather than a settled decision.

2026-08-29 — Do not copy replay buffers or TensorBoard logs into the repo

Chose: keep every reward curve, config and the *final* checkpoint per run; exclude `buffers/*.npy` and `events.out.tfevents*`.

Why: `results/` was 5.3 GB, dominated by replay buffers (regenerable training state) and TensorBoard event files, with one 188 MB log exceeding GitHub's 100 MB file limit. The actual scientific content — reward curves and configs — is under 1 MB; final checkpoints add ~20 MB. Result: 617 MB → 42 MB, and a repo that clones in seconds on both machines.

Rules out: resuming training from a mid-run epoch on the other machine, and re-opening the interactive TensorBoard curves there. Both remain available on this machine and on the USB copy.

Reference 5 — the Gymnasium re-test in detail

RL_DT/_docs_legion/gpu-retest-2026-08-29.md - 81 lines

Backs R1. Its 'why the old and new numbers are not directly comparable' section is the part that matters — do not put the old and new tables side by side in a slide.

GPU re-test of the previously validated environments — 2026-08-29

Re-ran TorchTD3-v0 on the environments recorded in PUBLICATION_MATERIALS.md, on the trossen-ai RTX 5090, after the TF/triton fix (../_patches/README.md) made the GPU actually usable.

Why the old and new numbers are not directly comparable

Three things differ from the published runs, and all of them matter:

- 1. **The old runs were CPU-bound.** Before the fix, importing TensorFlow broke torch's CUDA detection, so the torch agent silently fell back to Device: cpu even on a GPU machine. The published numbers came from Dewei's CPU-only laptop anyway, but the mechanism is worth stating: any pre-fix "GPU" run was not using the GPU.
- 2. **The driver seeds from wall-clock time** (run_continuous.py), so no run is reproducible, old or new. Single seed per arm, as the roadmap already flags.
- 3. **Different machine, different library versions.** Notably gymnasium-robotics is 1.4.2 here.

Treat the table below as "the toolkit still trains correctly on this machine", not as a benchmark comparison.

Results

Matched each environment's original episode/step counts.

Environment	Episodes × steps	Final avg-20	Wall time
Pendulum-v1	100 × 200	~160.30	111 s
MountainCarContinuous-v0	100 × 999	~1.02	477 s
Hopper-v5	300 × 1000	187.10	71 s
AdroitHandDoor-v1	20 × 200	~42.31	23 s
FetchReachDense-Flat-v0	500 × 50	failed to start	—

All four successful runs reported Device: cuda.

The FetchReach failure — a library problem, not a toolkit problem

```
gymnasium_robotics/utils/mujoco_utils.py:142 in set_joint_qpos
    assert joint_type in (mujoco.mjtJoint.mjJNT_HINGE, mujoco.mjtJoint.mjJNT_SLIDE)
AssertionError
```

It fails in gymnasium-robotics' own Fetch setup code, before the toolkit is involved at all — the environment cannot even be constructed. Versions here: gymnasium-robotics 1.4.2, mujoco 3.12.0, gymnasium 1.3.0.

Confirmed Fetch-specific: AdroitHandDoor-v1 constructs and trains fine on the same install, so envs/robotics_flat.py and the flattening wrapper are not implicated. The whole Fetch family is affected.

Likely fixes, untested: pin an older gymnasium-robotics, or an older mujoco matching what 1.4.2 expects. Worth resolving if the Fetch results matter for publication, since it is the one environment from the published set that cannot currently be reproduced here.

A second version caveat, on AdroitHandDoor

gymnasium-robotics prints on import:

```
AdroitHandDoorDense-v1 ... reward functions were updated in v1.2.1 without an environment version update. Therefore, use gymnasium-robotics==1.2.0 for v1 reproducibility or use v2 in gymnasium-robotics>=1.4.3.
```

So the AdroitHandDoor reward function here may differ from the one behind the published numbers, with no version-id change to signal it. The ~42.31 above should not be compared against the published AdroitHandDoor figure without first pinning the same library version.

Dependency gap this exposed

gymnasium-robotics is in neither requirements.txt nor env.yaml, yet envs/robotics_flat.py depends on it and utests/test_registry.py::test_env fails without it. Same category as the pysindy / scikit-learn gap already noted in environment.md. Installed here as gymnasium-robotics==1.4.2; adding it (pinned) to requirements.txt would be a small, useful upstream contribution.

Where the runs live

`_toolkits/toolkit-isaacsim/results/<env-id>/index090_*` — reward curves (`results.npy`), configs, and the final checkpoint per run. Replay buffers and TensorBoard event files are excluded from the repo by size.

Reference 6 — four Isaac Lab built-in tasks

RL_DT/_docs_legion/isaac-lab-built-in-tests-2026-08-29.md - 68 lines

Backs R2. This is the evidence that the adapter is a general bridge rather than shaped around Martin's scene, which is the strongest reusability claim available.

Toolkit vs Isaac Lab's built-in environments — 2026-08-29

Question: is isaac_vec_adapter.py a genuine toolkit↔Isaac Lab bridge, or does it only happen to fit Martin's door task?

Answer: it generalises. Four built-in Isaac Lab tasks, spanning very different observation and action shapes, all driven by TorchTD3-v0 with no per-task code.

Results

--num_envs 16 --steps 30, RTX 5090.

Task	Obs	Actions	Mean reward	Throughput
Isaac-Cartpole-v0	4	1	-0.3505	659 env-steps/s
Isaac-Ant-v0	60	8	0.0376	440 env-steps/s
Isaac-Reach-Franka-v0	32	7	-0.2214	257 env-steps/s
Isaac-Open-Drawer-Franka-v0	31	8	0.6499	364 env-steps/s

All four ran on cuda. 4/4 passed.

Observation dimension ranges 4→60 and actions 1→8 across these tasks; the adapter handled every combination unchanged. That is the evidence the bridge is real rather than door-task-shaped.

The rewards are meaningless as performance figures — 30 steps of a barely-initialised policy. They are recorded only to show the reward path is live rather than returning zeros.

Isaac-Open-Drawer-Franka-v0 is the interesting one: NVIDIA's cabinet-opening task, structurally the same problem as Martin's door — articulated object, grasp a handle, pull it open. It is almost certainly what the door task was modelled on, and it is worth reading alongside ur5e_door_open_env_cfg.py to see which parts are NVIDIA convention and which are Martin's choices.

A harness bug worth recording

The first version of this test looped over all four tasks inside one Python process, calling env.close() then gym.make() for the next one. It hung for 7 hours 40 minutes — GPU pinned at 0% utilisation with 2.5 GB VRAM held, process sleeping, no output after Cartpole.

Isaac Sim does not reliably support tearing down an environment and creating another within a single session. The task loop belongs in the shell, not in Python.

Fixed by splitting into: - test_one_builtin.py — runs exactly one task, then exits - run_builtin_tests.sh — loops over tasks, one Isaac Sim process each, with a 600 s timeout per task so a hang costs minutes rather than hours

Same four tasks then completed in a few minutes.

Two lessons, both mine rather than the toolkit's: 1. Never loop over Isaac Sim environment creation in one process. 2. Always put a timeout on a long-running background job. The 7h40m was invisible because nothing would ever have stopped it.

A minor related quirk: even the single-task processes linger during shutdown after writing their results. Results are complete and correct before that point, so the wrapper's timeout handles it.

Practical use beyond this test

Isaac-Cartpole-v0 starts in seconds and has a 4-dimensional observation, which makes it a far better debugging loop for adapter changes than waiting on the full lab scene. When something breaks in the wiring, reproduce it there first.

Files

- digital_twin_models-DT_reorg/RL_CloudTesting/test_one_builtin.py
- digital_twin_models-DT_reorg/RL_CloudTesting/run_builtin_tests.sh
- isaac_vec_adapter.py — the adapter under test, unchanged from the door-task work

Reference 7 — the 1 to 128 environment sweep

RL_DT/_docs_legion/dt-via-isaacsim-2026-08-29.md - 68 lines

Backs R3. Its 'why parallelism matters beyond speed' argument is the one worth carrying into a talk: at one environment TD3 cannot practically reach its own warmup threshold.

Toolkit → digital twin via Isaac Lab + Docker — 2026-08-29

The Isaac Lab route, run cleanly in sequence and measured across environment counts. Counterpart to `dt-via-isaacsim-direct` (the bare Isaac Sim route, `_toolkits/toolkit-isaacsim/`).

Setup

- Container `isaac-lab-base` (Isaac Sim 5.1.0 + Isaac Lab 2.3.2), RTX 5090
- Martin's assets mounted at `envs_external/`, toolkit at `external_projects/`
- Task `Isaac-UR5e-DoorOpen-v0`, obs 43, actions 7 (6 UR5e arm joints + gripper)
- `TorchTD3-v0` via `isaac_vec_adapter`. `IsaacVecBridge`, 2 episodes × 40 steps each

Scaling results

--num_envs	Env-steps/s (ep 0, ep 1)	Speed-up vs 1 env
1	15, 17	baseline
16	191, 180	~11×
64	419, 134	~9–25×
128	271, 181	~12–17×

All runs reported `TorchTD3` on `cuda`. No timeouts, no errors.

Throughput rises steeply to 16 environments, then flattens. That plateau is expected and is explained by a known limitation of the bridge: the simulator steps all N environments together on the GPU, but **action selection is still one forward pass per environment** in a Python loop. Past a certain N, that loop — not the physics — dominates. Removing it needs a `batch_action` method on the agent, which is a real change to Malachi's code rather than a wrapper. The per-episode variance (419 then 134 at 64 envs) is large enough that these numbers should be read as "roughly an order of magnitude, then flat", not as precise measurements. Two episodes per configuration is not enough to average out shader-compilation and cache effects on first touch.

Why parallelism matters here beyond speed

`cfigs/torch_td3.cfg` sets `warmup_size: 2500` — TD3 takes random actions until 2500 transitions are buffered, and only then begins learning. At 1 environment a 40-step episode contributes 40 transitions; at 128 it contributes 5120. **The single-environment configuration cannot realistically reach the point where training starts**, which is why its rewards stay flat regardless of how long it runs. This is the substantive argument for the vectorised bridge, more than the raw throughput.

Rewards

Mean reward sits around 1.5–1.9 across all configurations, with no trend. That is expected: these are 2-episode runs of an essentially untrained policy. **Nothing here demonstrates learning**, and no claim about learning should be made from this table. It demonstrates that the data path works end to end at every scale tested.

Files

Copied into `_toolkits/toolkit-isaacsim/isaac_integration/` so the tracked copy is self-contained:

File	Role
<code>isaac_gym_adapter.py</code>	single-env adapter (<code>num_envs=1</code> only)
<code>isaac_vec_adapter.py</code>	vectorised bridge — N envs step together, N transitions fanned out
<code>train_toolkit_td3.py</code>	single-env training script
<code>train_toolkit_td3_vec.py</code>	vectorised training script (used here)
<code>probe_door_env.py</code>	diagnostic — dumps the env's real shapes, dtypes and devices
<code>run_dt_scaling.sh</code>	the scaling sweep that produced the table above

One fix that made this possible

Martin's `ur5e_door_open_env_cfg.py` referenced the scene USD by bare filename (`TrainingScene_flattened.usd`) while the file lives in a sibling directory. Changed to `../ENV_OT_UR_door/TrainingScene_flattened.usd`. The line sits under a comment reading "User-editable paths", so adjusting it per setup appears to be expected rather than a defect.